Database Management PBL

Project title: Activity Points Ledger

▼ General info

VTU Activity Point Management, Document Mapping and Verification System

1. Short description

The **VTU Activity Point Management, Document Mapping and Verification System** is a role-based web application designed to digitize the college's Activity Point workflow from **event creation to official point awarding, evidence storage, verification, correction, and long-term audit**.

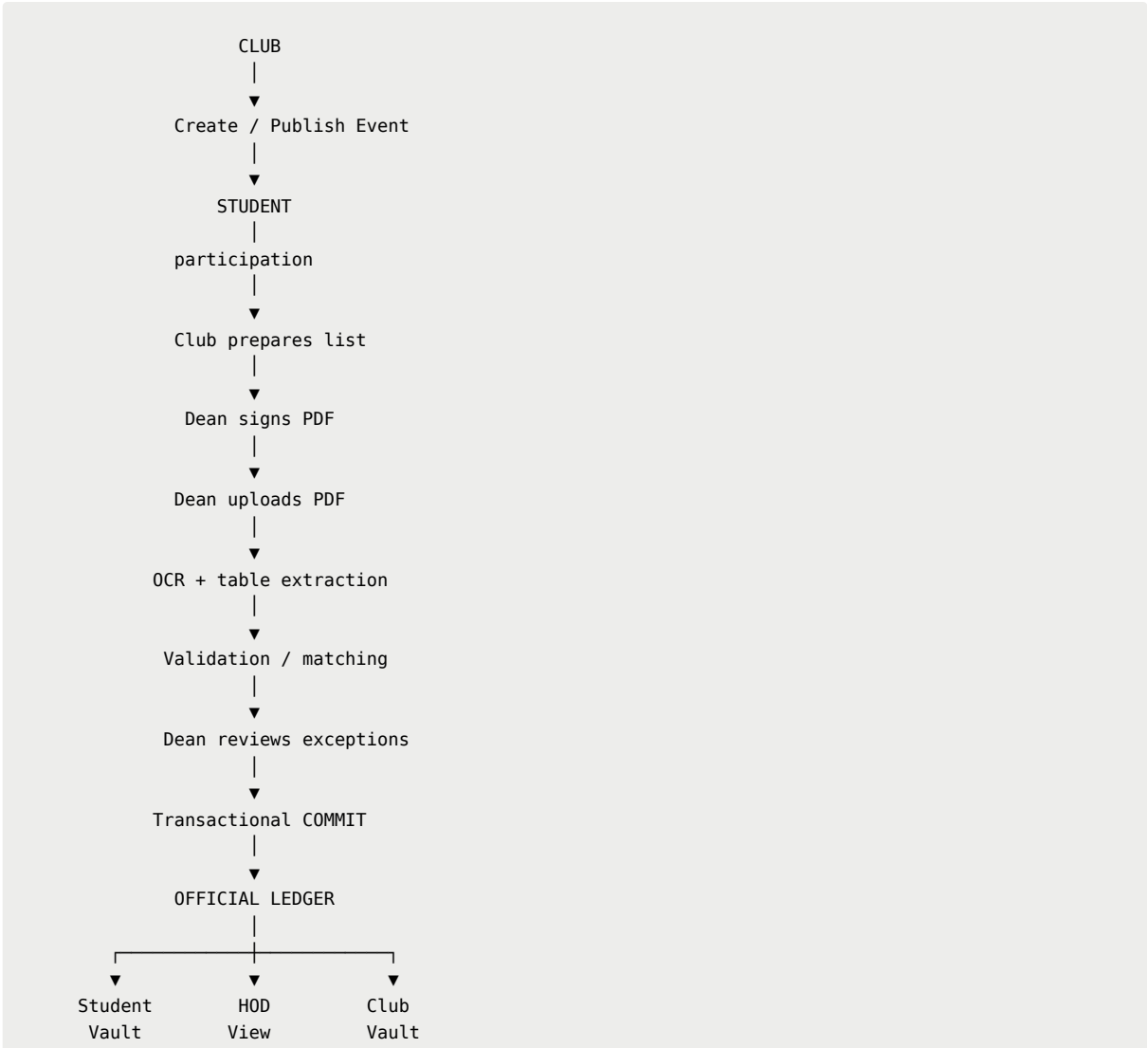
The system maintains a centralized **official Activity Point Ledger** for every student while separately managing:

- Club-created events visible to students
- Official signed point-award documents uploaded by the Dean
- OCR-based extraction of student/USN/point information
- Human verification of OCR results by the Dean
- Individual student certificates and supporting evidence
- Missing-point and correction requests
- Club-level verification
- Dean-level approval
- Student, Club, Dean and HOD document vaults
- Audit trails
- Department-level reporting
- Historical records of graduated students

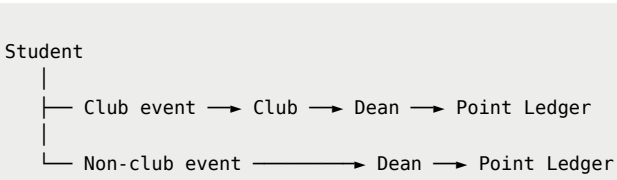
The core principle is:

> **An event, a point request, a point award, and a document are different things and are therefore represented separately in the database.**

The official workflow is:



Student-submitted claims follow a separate workflow:



---

2. Users and roles

Student

Can:

- Log in
- View own Activity Points
- View required points
- View remaining points
- View event history
- View upcoming events
- View official evidence
- Upload certificates
- Upload supporting evidence
- Tag documents to events
- Store archive documents
- Submit new activity requests
- Submit missing-from-list requests
- Submit correction requests
- Track request status
- View approved/rejected requests
- View documents associated with their awards

Club Lead

Can:

- Log in
- Create/manage club events
- Publish event information
- View their club's events
- View student issues for their events
- Review missing-from-list requests
- Review correction requests
- Review student evidence
- Verify/reject requests
- Forward verified requests to Dean

- View approved documents for their events
- Monitor unresolved issues

Cannot:

- Directly award Activity Points
- Modify the official ledger
- Approve their own final point awards
- Access unrelated clubs
- Create arbitrary correction records

Dean

The Dean is the **final authority for official Activity Point awards**.

Can:

- View all events
- Upload signed official documents
- Process OCR
- Review OCR results
- Resolve OCR exceptions
- Approve/reject student requests
- Commit official points
- View all students
- View all documents
- View club accountability
- View pending issues
- Upload additional signed correction documents
- Import student batches
- Generate reports
- Manage historical records
- Access audit information

HOD

HOD is primarily **read-only**.

Can:

- View students in their department
- Search students
- View Activity Points
- View event history
- View supporting documents
- View official signed documents
- View student-uploaded evidence
- View request status
- Generate department reports
- Export department data

Cannot:

- Award points
- Modify ledger
- Approve claims
- Access students from another department

---

3. Extensive feature list

A. Authentication and account management

Login

- Single login page
- College email + password
- No role-selection login
- Backend determines role
- Automatic dashboard redirection
- Logout
- Session/token management
- Account status

- Password hashing
- Password validation
- Authentication failure handling

Authorization

- Role-based access control
- Student-level authorization
- Club-level authorization
- Department-level authorization
- Dean-level global authorization
- Backend authorization
- API-level authorization
- Document-level authorization
- Prevent URL/object manipulation to access unauthorized records

---

B. Student dashboard

Displays:

- Name
- USN
- Department
- Entry type
- Admission year
- Current semester
- Student status
- Required Activity Points
- Earned Activity Points
- Remaining Activity Points
- Target reached/not reached
- Progress indicator
- Recent awards
- Pending requests

- Recently uploaded documents
- Upcoming events

Point calculation

Regular:

```
100 points
```

Lateral:

```
75 points
```

These are maintained through `POINT_REQUIREMENT`, not hardcoded into the frontend.

Students can exceed the target.

C. Student Activity Point ledger

Student can see:

- Event name
- Event date
- Points awarded
- Award date
- Source of award
- Approval status
- Supporting documents
- Official signed document
- Historical corrections
- Superseded awards

The total is calculated from active `POINT_LEDGER` records rather than stored redundantly in `STUDENT`.

D. Student Events page

Shows events created/published by clubs.

Each event can display:

- Event name
- Description
- Organizing club
- Date
- Start time
- End time
- Venue
- Registration URL
- Expected/standard points
- Academic year
- Event status

Important distinction

The Events page means:

> **"This activity/event exists."**

It does **not** mean:

> **"These students have been officially awarded points."**

Official point awarding happens through the separate signed-document/OCR/verification workflow.

E. Club event management

Club Lead can:

- Create event
- Edit event before appropriate locking/closure
- Add event name
- Add description
- Set event date
- Set venue
- Add registration link

- Set standard points
- Publish event
- View past events
- View event status
- View issues related to event

F. Official signed-document workflow

Dean uploads:

```
Signed Activity Point PDF
```

The system:

1. Stores the original document
2. Associates it with an event
3. Processes OCR
4. Extracts event information
5. Extracts club information
6. Extracts student names
7. Extracts USNs
8. Extracts points
9. Detects table rows
10. Matches USNs to students
11. Validates points
12. Detects exceptions
13. Generates verification report
14. Allows Dean to correct/resolve exceptions
15. Waits for explicit Dean confirmation
16. Performs transactional commit
17. Creates official ledger records
18. Links the official document to the resulting awards
19. Makes the approved document available through relevant vaults

G. OCR event/club matching

Dean should **not normally have to manually type event name and club**.

OCR attempts to extract:

```
Event:
TEDxRVCE SIP 2026

Club:
TEDxRVCE
```

The system attempts to match those against existing database records.

The Dean sees something like:

```
Detected Event: TEDxRVCE SIP 2026
Matched Event: TEDxRVCE SIP 2026       ✓

Detected Club: TEDxRVCE
Matched Club: TEDxRVCE                 ✓
```

If matching is uncertain:

```
⚠ Event match requires verification
[ Select existing event ]
```

Thus:

> **OCR proposes → database validates/matches → Dean confirms.**

H. OCR verification

System detects:

- Unknown USN
- Invalid USN format
- Name mismatch
- Invalid point value
- Missing point value
- Duplicate student row

- Duplicate award
- Low OCR confidence
- Unmatched event
- Unmatched club
- Unexpected table structure
- Missing required information

I. OCR verification report

Example:

```
Pages processed:       56
Records detected:      487
Valid records:         475
Unknown USNs:            4
Name mismatches:         3
Point errors:            2
Possible duplicates:     3
Exceptions:             12
```

Dean can inspect each exception.

J. Human-in-the-loop OCR

OCR output is **never automatically considered an official Activity Point award**.

Instead:

```
OCR
 ↓
OCR_RECORD
 ↓
Validation
 ↓
Dean review
 ↓
Dean confirmation
 ↓
Transaction
 ↓
POINT_LEDGER
```

K. Transaction and rollback

Final approval occurs inside a database transaction.

```
BEGIN
    insert ledger records
    link documents
    update requests
    create audit records
COMMIT
```

If an error occurs:

```
ROLLBACK
```

Therefore the database cannot end up with half of a signed list processed as official points.

L. Student certificate upload

Students can upload:

- Certificate PDF
- Certificate image
- Attendance evidence
- Event photograph
- Supporting photograph
- Report
- Other evidence

Multiple documents can be attached to **one request**.

Example:

```
Request #501

 ├── Certificate.pdf
 ├── Attendance.jpg
 └── GuestPhoto.jpg
```

M. Certificate OCR

OCR can extract:

- Student name

- USN
- Event name
- Event date
- Activity points
- Other relevant text

The student can review the extracted information.

OCR does not itself award points.

N. Evidence-only upload

A student can upload a document even when points have **already been awarded**.

Example:

```
TEDxRVCE
10 points already awarded

Student uploads:
TEDx_Certificate.pdf
```

This:

- Does not create another award
- Does not create a duplicate request
- Can be tagged to the event
- Appears in the student's vault
- Can be viewed by the relevant Dean
- Can be viewed by the relevant HOD
- Can be available to the relevant Club Lead when appropriate

The physical file is stored only once.

O. Document tagging

When uploading a document:

```
Document:
Certificate.pdf

Type:
```

```
Certificate

Related Event:
[ TEDxRVCE SIP 2026 ]

Purpose:
Evidence
```

Event association is encouraged but can be optional for archive documents.

P. Document vaults

Student vault

Contains:

- Own certificates
- Own supporting evidence
- Official documents associated with their awards
- Pending-request documents
- Archive documents

Club vault

Contains:

- Event documents
- Approved signed documents
- Documents associated with their events
- Relevant student evidence where authorized

Dean vault

Contains institution-wide authorized documents.

HOD vault

Contains authorized documents relating to students in the HOD's department.

Important

These are **logical views of the same underlying files**.

The system does not create four copies of the PDF.

Q. Point request types

`NEW_ACTIVITY`

Student claims points for an activity not currently recorded.

`MISSING_FROM_LIST`

Student participated in a club event but their USN was absent from the official list.

`CORRECTION`

Student reports that an existing point record is incorrect.

R. Request workflows

Club event

```
Student
 ↓
Club
 ↓
Dean
 ↓
Approve / Reject
```

Non-club event

```
Student
 ↓
Dean
 ↓
Approve / Reject
```

Missing-from-list

```
Student
 ↓
Club
 ↓
Dean
```

Correction

```
Student
 ↓
```

```
Club
 ↓
Dean
```

S. Club accountability dashboard

Dean sees each club and its events with issue counts.

Example:

Club	Event	Open Club Issues	Pending Dean	Resolved
TEDxRVCE	SIP 2026	4	2	31
IEEE	Workshop	0	5	27

Issues counted:

- `MISSING_FROM_LIST`
- `CORRECTION`

while they are awaiting club action.

T. Dynamic issue count

The number is **not stored as a counter**.

It is calculated from request status.

Example:

```
PENDING_CLUB
```

means it contributes to the Club's open issue count.

When Club forwards it:

```
PENDING_CLUB
       ↓
PENDING_DEAN
```

the Club count decreases automatically.

When Dean resolves it:

```
PENDING_DEAN
       ↓
```

```
RESOLVED
```

the Dean's pending count also decreases.

U. Bulk offline resolution

If a Club has many issues, it can choose to handle them offline.

Example:

```
25 missing students
       ↓
Club prepares additional signed list
       ↓
Dean uploads signed document
       ↓
OCR
       ↓
Dean verifies
       ↓
Points committed
       ↓
Matching requests marked RESOLVED
```

The issue counts therefore update automatically.

V. Request tracking

Students can see:

- Request ID
- Event
- Request type
- Submitted date
- Current status
- Club comments
- Dean comments
- Documents
- Decision
- Decision date

W. Correction handling

Approved records are not casually overwritten.

Example:

```
Original:
Event X → 10 points

Correction approved:
Event X → 15 points
```

The original can become:

```
SUPERSEDED
```

while the corrected record becomes:

```
ACTIVE
```

This preserves history.

X. Student batch import

Dean can import:

- USN
- Name
- Email
- Department
- Entry type
- Admission year
- Semester

Validation checks:

- Duplicate USN
- Duplicate email
- Invalid department
- Invalid entry type

- Missing values
- Invalid semester
- Existing student

Y. Graduated student management

Students are not automatically deleted when they graduate.

```
ACTIVE
   ↓
GRADUATED
```

Their records remain available for historical/audit purposes.

The system can additionally provide a **controlled batch archival/permanent deletion function** for storage management, subject to Dean authorization.

Permanent deletion should show what will be affected before confirmation and must not blindly delete shared official documents.

Z. Reporting

Reports can include:

- Student Activity Point summary
- Students below target
- Students who reached target
- Department-wise status
- Event-wise points
- Club-wise events
- Club issue counts
- Pending requests
- Resolved requests
- Rejected requests
- OCR exceptions
- Student evidence
- Graduated-student records

- Audit information

AA. Export

Possible exports:

- CSV
- Excel
- PDF reports

AB. Audit logging

Track:

- Login-related actions
- Document upload
- Document deletion
- OCR processing
- OCR verification
- Point approval
- Point rejection
- Request submission
- Club verification
- Dean decisions
- Corrections
- Student imports
- Batch deletion
- Record changes

Audit entry can contain:

- User
- Action
- Entity
- Entity ID
- Timestamp

- Old values
- New values

---

4. Technology stack

Frontend

React + JavaScript

Purpose:

- Student dashboard
- Club dashboard
- Dean dashboard
- HOD dashboard
- Event pages
- Document vault
- OCR verification interface
- Request tracking
- Tables
- Filters
- Search
- Upload interfaces
- Progress indicators

Why React?

Your application is not a static website. Different users continuously interact with changing:

- tables
- statuses
- requests
- documents
- OCR results
- point totals

React is well suited to this dynamic UI.

5. Backend

Python + FastAPI

FastAPI handles:

- REST APIs
- Authentication
- Authorization
- RBAC
- Student operations
- Event operations
- Point requests
- Document upload
- Document access
- OCR integration
- Validation
- Dean approval
- Database transactions
- Audit logging

Why Python?

The OCR/document-processing component fits naturally into the Python ecosystem.

Why FastAPI?

It provides a clean API layer between:

```
React
   ↓
FastAPI
   ↓
PostgreSQL
```

and allows the OCR pipeline to remain in the same Python ecosystem.

6. RDBMS

PostgreSQL

PostgreSQL is the **authoritative source of truth**.

It stores:

- Students
- Departments
- Users
- Clubs
- Events
- Point requirements
- Point requests
- Point awards
- Document metadata
- OCR staging data
- Audit logs
- Relationships

Why PostgreSQL?

The application has many strong relationships:

```
Student → Point Ledger
Event → Point Ledger
Event → Document
Request → Document
Ledger → Document
Club → Event
Department → Student
User → Actions
```

It needs:

- Foreign keys
- Constraints
- Transactions
- Joins

- Aggregation
- Views
- Referential integrity
- ACID consistency

Therefore PostgreSQL is a natural fit.

---

7. OCR

PaddleOCR + PP-Structure

Used for:

- PDF/image text extraction
- Table recognition
- USN extraction
- Name extraction
- Point extraction
- Event/club extraction
- Structured document processing

The important innovation isn't merely using OCR.

It is:

```
OCR
+
Database validation
+
Exception detection
+
Human verification
+
Transactional commit
```

---

8. Object storage

Actual files are stored in **private cloud object storage**.

Examples:

- Supabase Storage

- S3-compatible storage
- Cloudflare R2

The database does **not** store the actual PDF/JPG binary.

PostgreSQL stores:

```
document_id
file_name
storage_key
file_hash
mime_type
file_size
event_id
uploaded_by
...
```

Object storage stores:

```
PDF
JPG
PNG
```

This keeps the relational database efficient and makes deployed storage practical.

---

9. Security stack

Security is implemented at multiple levels:

Authentication

- College email
- Password
- Argon2id/bcrypt password hashing

Authorization

- RBAC
- Object-level authorization
- Department-level authorization
- Club-level authorization
- Dean-only operations

Transport

- HTTPS/TLS

File security

- Private object storage
- File validation
- File size limits
- Safe filenames/storage keys
- SHA-256 hashes
- Authorized document access

API security

- Input validation
- Rate limiting
- Secure authentication
- CORS restrictions
- Secure cookies/tokens
- Dependency updates
- Error handling
- Audit logging

Database security

- Parameterized queries/ORM
- Least-privilege database access
- Backups
- Transaction management

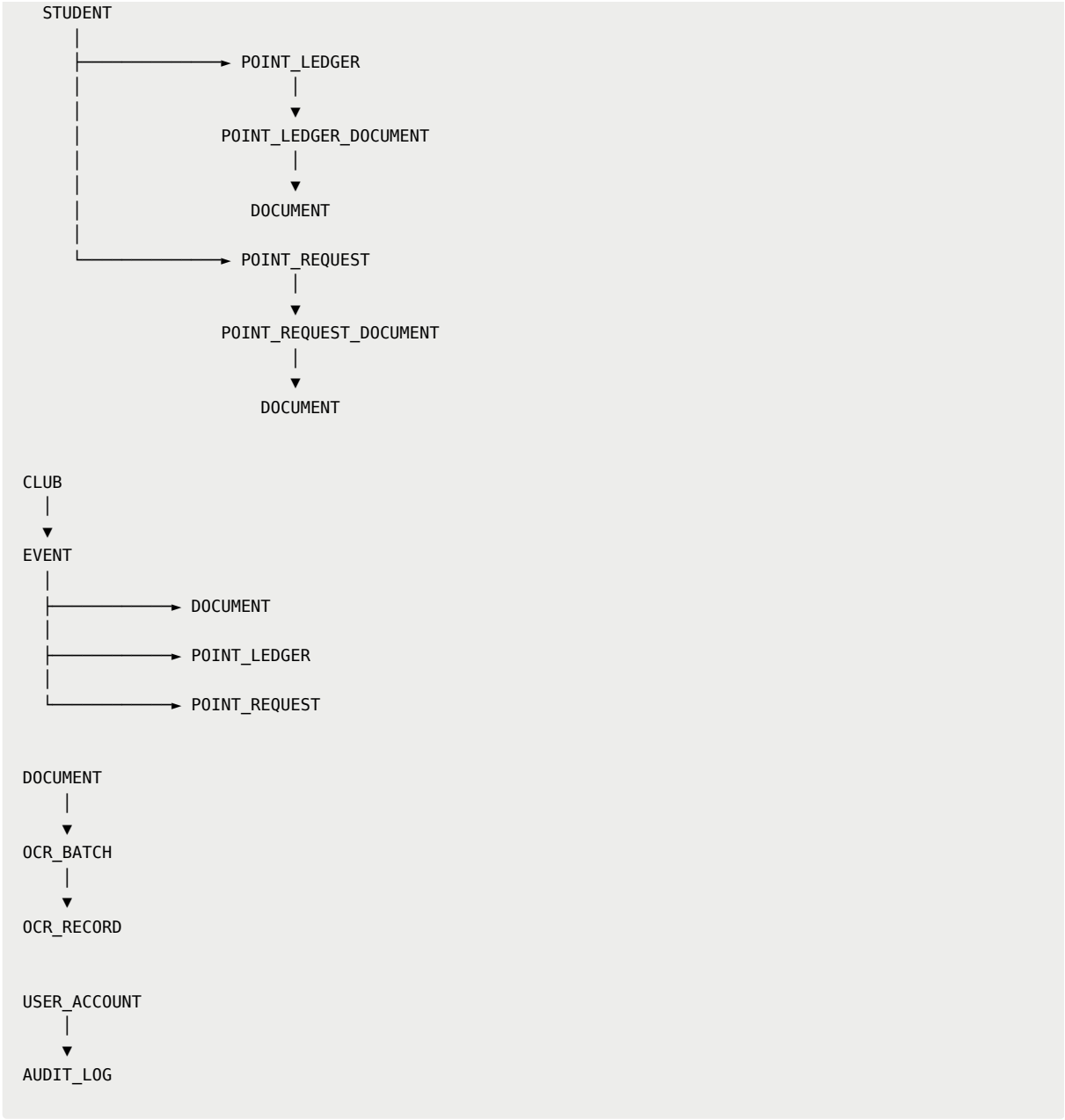
FastAPI itself is not "immune to cyberattacks"; the security comes from this complete architecture and secure deployment.

---

10. Relational Database Structure

The database is designed around the following major entities:

```
DEPARTMENT
     |
     ▼
```



11. RDBMS / NoSQL Integration

Final architecture: no separate NoSQL database

This is deliberate.

There is **no need to introduce MongoDB or another NoSQL database** simply because the project asks about RDBMS/NoSQL.

PostgreSQL remains the single authoritative structured database.

What about semi-structured OCR data?

PostgreSQL's `JSONB` is used.

For example, OCR may generate:

```
{
  "raw_text": "...",
  "layout_blocks": [],
  "detected_columns": [
    "Name",
    "USN",
    "Points"
  ],
  "ocr_metadata": {}
}
```

This can be stored in:

```
OCR_BATCH.extracted_metadata
OCR_RECORD.raw_extraction
```

Similarly, audit changes can use:

```
AUDIT_LOG.old_values
AUDIT_LOG.new_values
```

as `JSONB`.

Therefore:

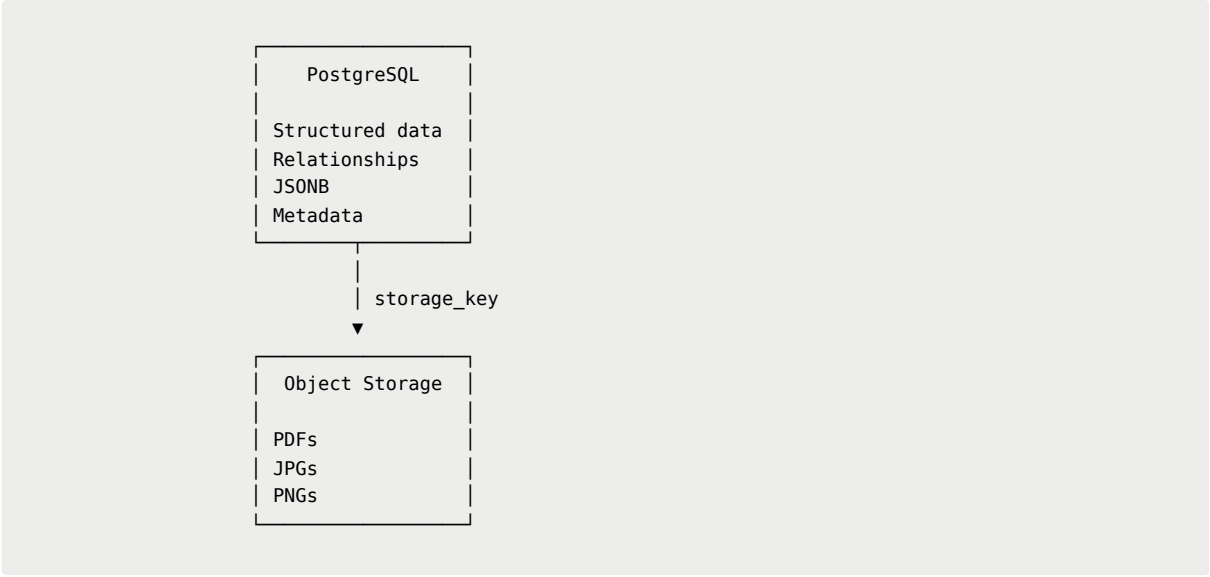
```
Structured relational data
          ↓
      PostgreSQL
          +
Semi-structured OCR/audit data
          ↓
     PostgreSQL JSONB
```

What about PDFs and images?

They are neither RDBMS nor NoSQL data.

They are **binary objects**, stored in cloud object storage.

Therefore:



There is no RDBMS–NoSQL synchronization problem.

If an academic requirement explicitly demands a NoSQL database, MongoDB could be added as an optional raw OCR/document-analysis store, but **it is not justified by the application's actual requirements**.

---

12. Complete table list

The finalized core relational model consists of:

1. `DEPARTMENT`
2. `STUDENT`
3. `USER_ACCOUNT`
4. `POINT_REQUIREMENT`
5. `CLUB`
6. `EVENT`
7. `DOCUMENT`
8. `OCR_BATCH`
9. `OCR_RECORD`
10. `POINT_REQUEST`
11. `POINT_REQUEST_DOCUMENT`
12. `POINT_LEDGER`
13. `POINT_LEDGER_DOCUMENT`

14. `AUDIT_LOG`

Plus database views such as:

```
STUDENT_POINT_SUMMARY
```

---

13. `DEPARTMENT`

Stores college departments.

Attribute	Data type	Constraints
`department_code`	VARCHAR(10)	**PK**, NOT NULL
`department_name`	VARCHAR(100)	NOT NULL, UNIQUE

Example:

```
CSE
ISE
ECE
ME
CV
```

We don't need a generated `department_id` because the college already has a stable department code.

---

14. `STUDENT`

Attribute	Data type	Constraints
`usn`	VARCHAR(20)	**PK**, NOT NULL
`name`	VARCHAR(150)	NOT NULL
`email`	VARCHAR(150)	NOT NULL, UNIQUE
`department_code`	VARCHAR(10)	FK → `DEPARTMENT`, NOT NULL
`entry_type`	VARCHAR(20)	NOT NULL, CHECK
`admission_year`	SMALLINT	NOT NULL
`current_semester`	SMALLINT	CHECK
`student_status`	VARCHAR(20)	NOT NULL, CHECK

`entry_type`

```
REGULAR
LATERAL
```

`student_status`

```
ACTIVE
GRADUATED
INACTIVE
ARCHIVED
```

`ARCHIVED` can be used if the institution chooses to archive old batches.

---

15. `USER_ACCOUNT`

We have simplified this based on your question.

Attribute	Data type	Constraints
`email`	VARCHAR(150)	**PK**, NOT NULL
`password_hash`	TEXT	NOT NULL
`role`	VARCHAR(20)	NOT NULL, CHECK
`account_status`	VARCHAR(20)	NOT NULL
`student_usn`	VARCHAR(20)	FK → `STUDENT`, nullable
`department_code`	VARCHAR(10)	FK → `DEPARTMENT`, nullable
`club_id`	BIGINT	FK → `CLUB`, nullable
`created_at`	TIMESTAMPTZ	NOT NULL
`last_login_at`	TIMESTAMPTZ	nullable

Roles

```
STUDENT
CLUB_LEAD
HOD
DEAN
```

Why email is PK

The college email is already the authentication identity.

We therefore do **not** need an artificial `user_id`.

---

16. `POINT_REQUIREMENT`

Attribute	Data type	Constraints
`entry_type`	VARCHAR(20)	**PK**
`required_points`	NUMERIC(6,2)	NOT NULL, CHECK > 0

Data:

```
REGULAR → 100
LATERAL → 75
```

---

17. `CLUB`

We've simplified this table.

Attribute	Data type	Constraints
`club_id`	BIGINT	**PK**, GENERATED IDENTITY
`club_name`	VARCHAR(150)	NOT NULL, UNIQUE
`club_email`	VARCHAR(150)	UNIQUE, nullable

We removed:

- Description — not necessary for core functionality
- Status — optional
- Created-at — optional

These can be added later if the actual college workflow requires them.

Why `club_id`?

The college doesn't need to already have a club ID.

This is an **internal database identifier**.

For example:

```
club_id = 17
club_name = TEDxRVCE
```

PostgreSQL generates `17`.

If the club name changes, relationships don't break.

---

18. `EVENT`

Attribute	Data type	Constraints
`event_id`	BIGINT	**PK**, GENERATED IDENTITY
`event_name`	VARCHAR(200)	NOT NULL
`club_id`	BIGINT	FK → `CLUB`, nullable
`description`	TEXT	nullable
`event_date`	DATE	NOT NULL
`start_time`	TIME	nullable
`end_time`	TIME	nullable
`venue`	VARCHAR(200)	nullable
`registration_url`	TEXT	nullable
`standard_points`	NUMERIC(6,2)	nullable, CHECK ≥ 0
`academic_year`	VARCHAR(20)	NOT NULL
`event_status`	VARCHAR(30)	NOT NULL
`created_by`	VARCHAR(150)	FK → `USER_ACCOUNT.email`
`created_at`	TIMESTAMPTZ	NOT NULL

Club event

```
club_id = 17
```

Non-club event

```
club_id = NULL
```

---

19. Why `event_id` instead of event name?

Because names aren't reliable unique identifiers.

For example:

```
Workshop
Workshop
Workshop
```

could occur in different years.

The database can instead have:

```
event_id = 1842
event_name = Workshop
date = 2026-03-10
```

The number is generated automatically by PostgreSQL:

```
event_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY
```

Users don't need to see it.

---

20. `DOCUMENT`

Attribute	Data type	Constraints
`document_id`	BIGINT	**PK**, GENERATED IDENTITY
`event_id`	BIGINT	FK → `EVENT`, nullable
`document_type`	VARCHAR(30)	NOT NULL, CHECK
`purpose`	VARCHAR(20)	NOT NULL, CHECK
`file_name`	VARCHAR(255)	NOT NULL
`storage_key`	TEXT	NOT NULL, UNIQUE
`mime_type`	VARCHAR(100)	NOT NULL
`file_size_bytes`	BIGINT	NOT NULL, CHECK > 0
`file_hash`	CHAR(64)	NOT NULL
`uploaded_by`	VARCHAR(150)	FK → `USER_ACCOUNT.email`
`uploaded_at`	TIMESTAMPTZ	NOT NULL
`document_status`	VARCHAR(20)	NOT NULL

Document types

```
SIGNED_POINT_LIST
CERTIFICATE
ATTENDANCE_SHEET
PHOTO
REPORT
OTHER
```

Purpose

```
OFFICIAL
EVIDENCE
ARCHIVE
```

`event_id` is nullable

Because:

```
Certificate → Event X
```

is possible.

But:

```
Old certificate → no event
```

is also possible.

---

21. `OCR_BATCH`

Represents an OCR processing operation.

Attribute	Data type	Constraints
`ocr_batch_id`	BIGINT	**PK**, GENERATED IDENTITY
`document_id`	BIGINT	FK → `DOCUMENT`, NOT NULL
`started_by`	VARCHAR(150)	FK → `USER_ACCOUNT.email`
`status`	VARCHAR(30)	NOT NULL
`started_at`	TIMESTAMPTZ	NOT NULL
`completed_at`	TIMESTAMPTZ	nullable
`pages_processed`	INTEGER	CHECK ≥ 0
`records_detected`	INTEGER	CHECK ≥ 0
`valid_records`	INTEGER	CHECK ≥ 0
`exception_count`	INTEGER	CHECK ≥ 0
`extracted_metadata`	JSONB	nullable
`error_message`	TEXT	nullable

---

22. `OCR_RECORD`

Stores individual OCR results before they become official.

Attribute	Data type	Constraints
`ocr_record_id`	BIGINT	**PK**, GENERATED IDENTITY
`ocr_batch_id`	BIGINT	FK → `OCR_BATCH`, NOT NULL
`page_number`	INTEGER	NOT NULL
`row_number`	INTEGER	nullable
`extracted_usn`	VARCHAR(20)	nullable
`matched_student_usn`	VARCHAR(20)	FK → `STUDENT`, nullable
`extracted_name`	VARCHAR(150)	nullable
`extracted_points`	NUMERIC(6,2)	nullable
`confidence_score`	NUMERIC(5,4)	CHECK 0–1
`raw_extraction`	JSONB	nullable
`validation_status`	VARCHAR(30)	NOT NULL
`validation_message`	TEXT	nullable

This is the staging area between OCR and the official ledger.

---

23. `POINT_REQUEST`

Attribute	Data type	Constraints
`request_id`	BIGINT	**PK**, GENERATED IDENTITY
`usn`	VARCHAR(20)	FK → `STUDENT`, NOT NULL
`event_id`	BIGINT	FK → `EVENT`, NOT NULL
`request_type`	VARCHAR(30)	NOT NULL, CHECK
`claimed_points`	NUMERIC(6,2)	CHECK ≥ 0
`reason`	TEXT	NOT NULL
`submitted_by`	VARCHAR(150)	FK → `USER_ACCOUNT.email`
`correction_of_ledger_id`	BIGINT	FK → `POINT_LEDGER`, nullable
`status`	VARCHAR(30)	NOT NULL
`club_reviewed_by`	VARCHAR(150)	FK → `USER_ACCOUNT.email`, nullable
`club_reviewed_at`	TIMESTAMPTZ	nullable
`club_review_comment`	TEXT	nullable
`dean_decided_by`	VARCHAR(150)	FK → `USER_ACCOUNT.email`, nullable
`dean_decided_at`	TIMESTAMPTZ	nullable
`dean_decision_comment`	TEXT	nullable

Attribute	Data type	Constraints
`created_at`	TIMESTAMPTZ	NOT NULL
`updated_at`	TIMESTAMPTZ	NOT NULL

Request types

```
NEW_ACTIVITY
MISSING_FROM_LIST
CORRECTION
```

Status

```
PENDING_CLUB
PENDING_DEAN
APPROVED
REJECTED
RESOLVED
WITHDRAWN
```

---

24. `POINT_REQUEST_DOCUMENT`

Many documents can support one request.

Attribute	Data type	Constraints
`request_id`	BIGINT	FK → `POINT_REQUEST`
`document_id`	BIGINT	FK → `DOCUMENT`
`attached_at`	TIMESTAMPTZ	NOT NULL

Primary key:

```
(request_id, document_id)
```

Relationship:

```
POINT_REQUEST
      |
      ├── Certificate
      ├── Attendance Photo
      └── Guest Photo
```

---

25. `POINT_LEDGER`

This is the official Activity Point record.

Attribute	Data type	Constraints
`ledger_id`	BIGINT	**PK**, GENERATED IDENTITY
`usn`	VARCHAR(20)	FK → `STUDENT`, NOT NULL
`event_id`	BIGINT	FK → `EVENT`, NOT NULL
`points_awarded`	NUMERIC(6,2)	NOT NULL, CHECK ≥ 0
`award_date`	DATE	NOT NULL
`source_type`	VARCHAR(30)	NOT NULL
`source_request_id`	BIGINT	FK → `POINT_REQUEST`, nullable
`source_ocr_batch_id`	BIGINT	FK → `OCR_BATCH`, nullable
`approved_by`	VARCHAR(150)	FK → `USER_ACCOUNT.email`
`approved_at`	TIMESTAMPTZ	NOT NULL
`status`	VARCHAR(20)	NOT NULL
`supersedes_ledger_id`	BIGINT	FK → `POINT_LEDGER`, nullable
`created_at`	TIMESTAMPTZ	NOT NULL

Source types

```
BULK_APPROVAL
INDIVIDUAL_CLAIM
MANUAL_CORRECTION
```

Status

```
ACTIVE
SUPERSEDED
REVOKED
```

---

26. Why `ledger_id`?

Unlike a student's USN, the point award itself does not have an existing real-world unique identifier.

A student could have:

```
Event A → 10 points
Event B → 15 points
Event C → 10 points
```

Each award needs to be individually referenced.

Therefore:

```
ledger_id
```

is useful for:

- Corrections
- Audit
- Document relationships
- Identifying a specific award
- Superseding an award

PostgreSQL generates it automatically.

---

27. `POINT_LEDGER_DOCUMENT`

Links official point awards with supporting documents.

Attribute	Data type	Constraints
`ledger_id`	BIGINT	FK → `POINT_LEDGER`
`document_id`	BIGINT	FK → `DOCUMENT`
`linked_at`	TIMESTAMPTZ	NOT NULL
`linked_by`	VARCHAR(150)	FK → `USER_ACCOUNT.email`

Primary key:

```
(ledger_id, document_id)
```

This allows:

```
ONE AWARD
   ↓
Certificate
Attendance
```

```
Photograph
Official signed PDF
```

and:

```
ONE OFFICIAL PDF
   ↓
Student A award
Student B award
Student C award
...
```

---

28. `AUDIT_LOG`

Attribute	Data type	Constraints
`audit_id`	BIGINT	**PK**, GENERATED IDENTITY
`user_email`	VARCHAR(150)	FK → `USER_ACCOUNT.email`
`action`	VARCHAR(100)	NOT NULL
`entity_type`	VARCHAR(50)	NOT NULL
`entity_id`	BIGINT	NOT NULL
`old_values`	JSONB	nullable
`new_values`	JSONB	nullable
`ip_address`	INET	nullable
`created_at`	TIMESTAMPTZ	NOT NULL

Example:

```
User: dean@rvce.edu.in
Action: APPROVE_OCR_BATCH
Entity: OCR_BATCH
Entity ID: 42
Time: ...
```

---

29. Important relationships

Department → Student

```
DEPARTMENT 1 ——— N STUDENT
```

Student → Point Ledger

```
STUDENT 1 ——— N POINT_LEDGER
```

Event → Point Ledger

```
EVENT 1 ——— N POINT_LEDGER
```

Therefore:

```
STUDENT N ——— N EVENT
```

is represented by `POINT_LEDGER`.

Club → Event

```
CLUB 1 ——— N EVENT
```

For non-club events:

```
EVENT.club_id = NULL
```

Event → Document

```
EVENT 1 ——— N DOCUMENT
```

But a document can exist without an event.

Document → OCR Batch

```
DOCUMENT 1 ——— N OCR_BATCH
```

This allows OCR processing to be repeated if necessary.

OCR Batch → OCR Record

```
OCR_BATCH 1 ──── N OCR_RECORD
```

Student → Point Request

```
STUDENT 1 ──── N POINT_REQUEST
```

Event → Point Request

```
EVENT 1 ──── N POINT_REQUEST
```

Request ↔ Document

```
POINT_REQUEST
      N
      |
      |
POINT_REQUEST_DOCUMENT
      |
      |
      N
      |
DOCUMENT
```

Many-to-many.

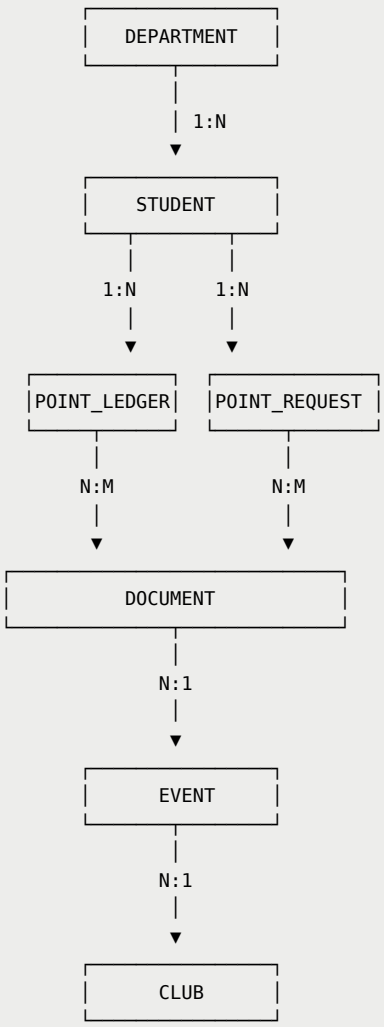
Point Ledger ↔ Document

```
POINT_LEDGER
      N
      |
POINT_LEDGER_DOCUMENT
      |
      N
      |
DOCUMENT
```

Many-to-many.

This is one of the most important relationships in the system.

30. Complete relationship picture




```
DOCUMENT
    │
    │ 1:N
    ▼
OCR_BATCH
    │
    │ 1:N
    ▼
OCR_RECORD


USER_ACCOUNT
    │
    │ 1:N
    ▼
AUDIT_LOG
```

31. Why the database does NOT contain several tempting tables

We've deliberately removed several things that might initially seem useful.

No `ACTIVITY`

Because **EVENT is the single representation of a point-earning occurrence**.

No `EVENT_PARTICIPATION`

Because this isn't an attendance/registration-management system.

The official signed list is the authoritative source for participation/points.

No `STUDENT_CLUB_MEMBERSHIP`

Club membership has no role in calculating Activity Points.

No `AICTE_CATEGORY`

Not required by the actual system requirements.

No `STUDENT.total_points`

Total is derived from the ledger.

No `CLUB.open_issue_count`

It is derived from `POINT_REQUEST.status`.

No `document_id` directly inside `POINT_LEDGER`

Because one award can have multiple documents and one official document can support many awards.

No physical copy of a document for every vault

The file exists once in object storage. Different users see it through authorized database relationships.

---

32. Innovative component

The strongest innovation is not simply:

> "We use OCR."

It is the complete **OCR-assisted, human-verified document-to-ledger pipeline**.

Innovation 1 — Existing signed documents become structured database records

The college can continue producing its familiar signed documents.

Instead of manually typing hundreds of student records:

```
Signed PDF
    ↓
OCR
    ↓
USN / Name / Points
    ↓
Database
```

Innovation 2 — Human-in-the-loop verification

The system does not blindly trust OCR.

```
OCR
 ↓
Validation
 ↓
Exception detection
 ↓
Dean verification
 ↓
Transactional commit
```

This is particularly important because Activity Points are academic records.

Innovation 3 — Document-to-ledger traceability

A student's point can be traced through:

```
Student
   ↓
Point Award
   ↓
Event
   ↓
Supporting Document
   ↓
Original Signed PDF
   ↓
OCR Record
   ↓
Dean Verification
```

Innovation 4 — Multiple evidence without duplicate awards

The system distinguishes:

```
DOCUMENT
POINT REQUEST
POINT AWARD
```

So:

```
Certificate
+
Attendance photograph
+
Official signed list
```

can all support the same award without accidentally awarding the points multiple times.

Innovation 5 — Existing-award evidence preservation

Students can add certificates **after the points have already been awarded**.

The certificate:

- becomes part of the evidence record
- is associated with the event
- is visible to authorized Dean/HOD users
- does not create another award

This solves the long-term evidence problem.

Innovation 6 — Automated club accountability

The Dean dashboard doesn't merely show requests.

It provides an event/club-level accountability view:

```
Club
 ↓
Event
 ↓
Number of unresolved issues
 ↓
Club forwards/resolves issues
 ↓
Count automatically changes
```

This gives the college visibility into where the Activity Point workflow is getting stuck.

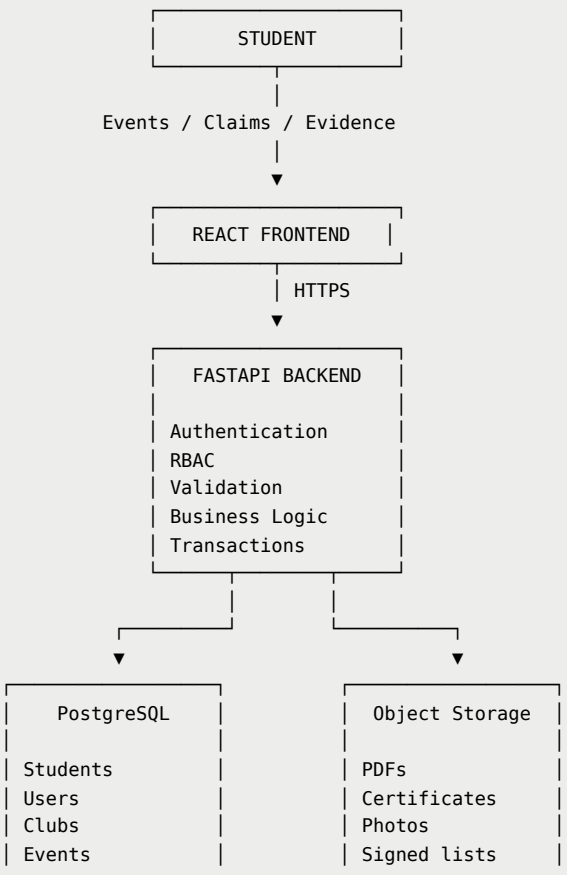
Innovation 7 — Transaction-safe bulk processing

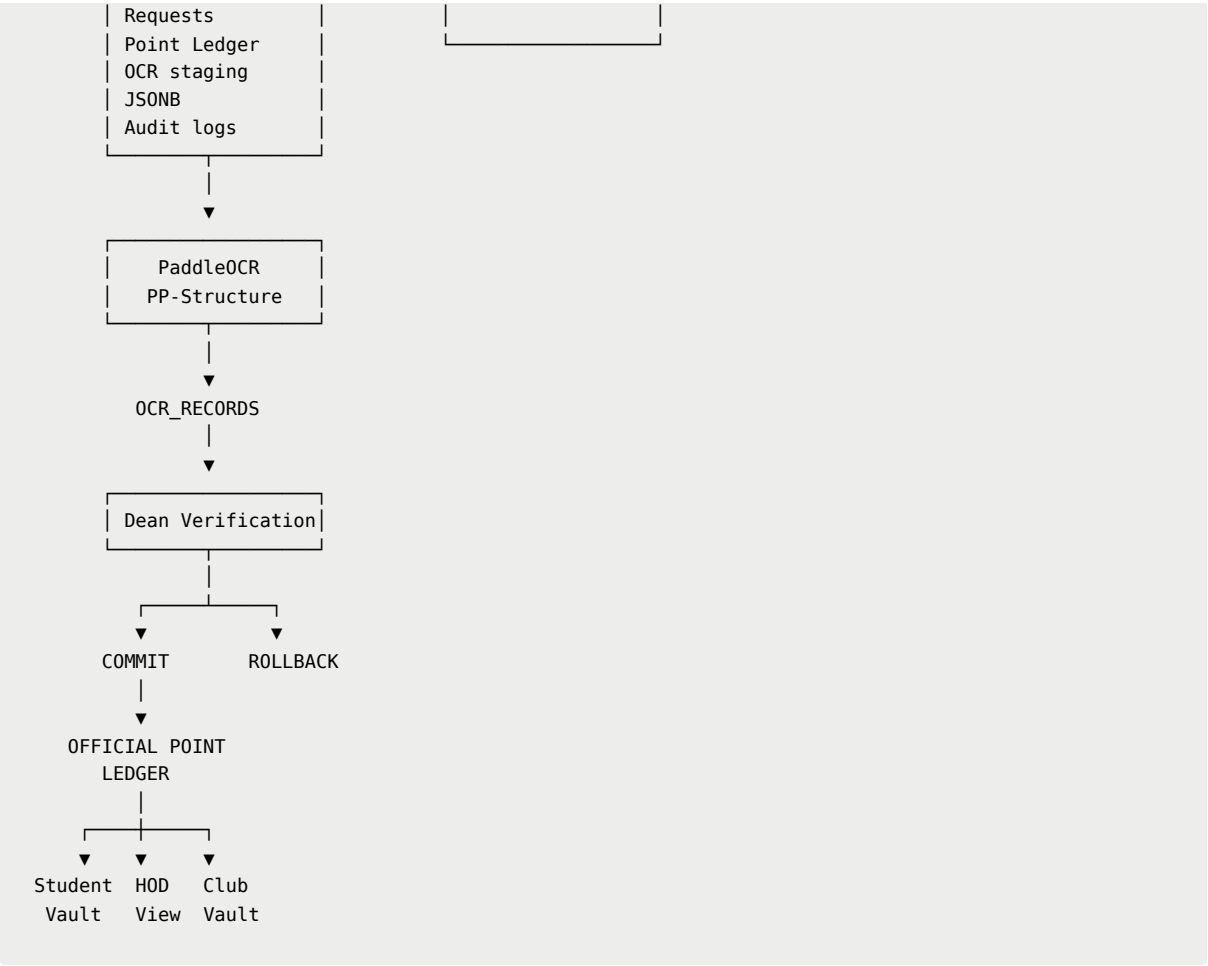
A 50+ page signed document containing hundreds of students can be processed without partially corrupting the official ledger.

```
OCR staging
      ↓
Verification
      ↓
BEGIN TRANSACTION
      ↓
Commit all
      OR
Rollback all
```

33. The final conceptual architecture

The entire system can ultimately be summarized as:





The central architectural idea

> **The database is the source of truth for points and relationships; cloud storage is the source of truth for the actual documents; OCR is an extraction assistant; and the Dean remains the authority that converts extracted information into official academic records.**

That separation is what makes the project both a strong **DBMS project** and a plausible **real college deployment**, rather than simply a CRUD application with an OCR feature.

▼ Tables + attributes

Yes. At this point, I would **freeze the ER model** rather than keep adding fields.

I went through the architecture again with one specific question:

> **Does every table and every attribute contribute something that the system actually needs?**

Where an attribute can be derived reliably from another attribute/table, I have removed it. Where it looks redundant but is needed for a different workflow, I have kept it.

Final answer: 14 tables

I recommend that your ER diagram contain **exactly 14 tables**:

1. `DEPARTMENT`
2. `STUDENT`
3. `USER_ACCOUNT`
4. `POINT_REQUIREMENT`
5. `CLUB`
6. `EVENT`
7. `DOCUMENT`
8. `OCR_BATCH`
9. `OCR_RECORD`
10. `POINT_REQUEST`
11. `POINT_REQUEST_DOCUMENT`
12. `POINT_LEDGER`
13. `POINT_LEDGER_DOCUMENT`
14. `AUDIT_LOG`

And **one view**, which is **not a table**:

1. `STUDENT_POINT_SUMMARY` — database view

I would **not add any other table** at this stage.

---

1. DEPARTMENT

Purpose

Stores the college's departments.

We need this because:

- Students belong to departments.
- HOD access is department-specific.
- Department reports are required.
- HOD must not see students from another department.

Attributes

Attribute	Type	Constraints	Why needed
`department_code`	`VARCHAR(10)`	**PK**, NOT NULL	Existing college identifier such as CSE, ISE, ME, etc.
`department_name`	`VARCHAR(100)`	NOT NULL, UNIQUE	Human-readable department name

Final table

```
DEPARTMENT
-----------
department_code       PK
department_name       UNIQUE
```

Anything redundant?

No.

Why no `department_id`?

Because `CSE`, `ISE`, `ME`, etc. are already stable identifiers.

Adding:

```
department_id = 1
department_code = CSE
```

would add an unnecessary layer.

---

2. STUDENT

Purpose

Stores the academic identity and current lifecycle state of every student.

This is one of the core tables.

Attributes

Attribute	Type	Constraints	Why needed
`usn`	`VARCHAR(20)`	**PK**, NOT NULL	Unique student identifier and central reference throughout system
`name`	`VARCHAR(150)`	NOT NULL	Student identity; also used for OCR matching

Attribute	Type	Constraints	Why needed
`email`	`VARCHAR(150)`	NOT NULL, UNIQUE	Student login/account association and communication
`department_code`	`VARCHAR(10)`	FK → `DEPARTMENT`, NOT NULL	Determines department and HOD visibility
`entry_type`	`VARCHAR(20)`	NOT NULL, CHECK	Determines 100 vs 75 point requirement
`admission_year`	`SMALLINT`	NOT NULL	Batch identification and historical management
`current_semester`	`SMALLINT`	CHECK	Student's current academic semester
`student_status`	`VARCHAR(20)`	NOT NULL, CHECK	ACTIVE / GRADUATED / INACTIVE / ARCHIVED

Final table

```
STUDENT
-------
usn                     PK
name
email                   UNIQUE
department_code         FK
entry_type
admission_year
current_semester
student_status
```

Why `entry_type`?

Because:

```
REGULAR  → 100
LATERAL  → 75
```

and this connects to `POINT_REQUIREMENT`.

Why `student_status`?

Because a graduated student must **not disappear from the system**.

```
ACTIVE
   ↓
GRADUATED
```

Historical records remain available.

Why `current_semester`?

The college dashboard/import/reporting needs to know the student's current academic position.

Is `current_semester` derivable from admission year?

Technically, potentially yes.

But because:

- students can have different academic progression,
- lateral students enter at a different stage,
- academic progression may not always be perfectly synchronized,

I would **keep it as maintained academic data** rather than calculate it blindly.

Is `email` redundant with `USER_ACCOUNT.email`?

There is some duplication, but **keep it**.

`STUDENT.email` is academic/student information.

`USER_ACCOUNT.email` is the authentication identity.

They are linked through the user's account, and keeping the student email allows batch import and student records to exist independently of whether an account has been created.

---

3. USER_ACCOUNT

Purpose

Stores authentication and authorization identity.

We have now simplified this considerably.

Attributes

Attribute	Type	Constraints	Why needed
`email`	`VARCHAR(150)`	**PK**, NOT NULL	Login identity
`password_hash`	`TEXT`	NOT NULL	Secure password storage
`role`	`VARCHAR(20)`	NOT NULL, CHECK	Determines Student/Club Lead/HOD/Dean permissions
`account_status`	`VARCHAR(20)`	NOT NULL, CHECK	Enables/disables account
`student_usn`	`VARCHAR(20)`	FK → `STUDENT`, NULL	Associates Student account with student
`department_code`	`VARCHAR(10)`	FK → `DEPARTMENT`, NULL	Associates HOD with department
`club_id`	`BIGINT`	FK → `CLUB`, NULL	Associates Club Lead with club

Attribute	Type	Constraints	Why needed
`created_at`	`TIMESTAMPTZ`	NOT NULL	Account creation/audit information
`last_login_at`	`TIMESTAMPTZ`	NULL	Useful account-security information

Final table

```
USER_ACCOUNT
------------
email                    PK
password_hash
role
account_status
student_usn              FK, NULL
department_code          FK, NULL
club_id                  FK, NULL
created_at
last_login_at
```

Why are the last three nullable?

Because different roles need different associations.

Student

```
student_usn = 1RV...
department_code = NULL
club_id = NULL
```

The department can already be obtained through the student.

HOD

```
student_usn = NULL
department_code = ISE
club_id = NULL
```

Club Lead

```
student_usn = NULL
department_code = NULL
club_id = 17
```

Dean

```
student_usn = NULL
department_code = NULL
club_id = NULL
```

Is this perfectly normalized?

It's acceptable for this project, but there is a role-dependent constraint:

> `student_usn` should be populated for STUDENT, `department_code` for HOD, and `club_id` for CLUB_LEAD.

That can be enforced by application logic and/or database checks.

Why no `user_id`?

Because you specifically have a stable identifier:

> **college email**

So:

```
email PK
```

is sufficient.

---

4. POINT_REQUIREMENT

Purpose

Stores the Activity Point requirement according to entry type.

Attributes

Attribute	Type	Constraints	Why needed
`entry_type`	`VARCHAR(20)`	**PK**	REGULAR/LATERAL
`required_points`	`NUMERIC(6,2)`	NOT NULL, CHECK > 0	Required target

Final table

```
POINT_REQUIREMENT
-----------------
entry_type          PK
required_points
```

Data:

```
REGULAR  → 100
LATERAL  → 75
```

Anything redundant?

No.

Why not store required points in STUDENT?

Because it is determined by `entry_type`.

Storing it in both places would create duplication.

---

5. CLUB

Purpose

Stores the clubs that create/manage events.

Attributes

Attribute	Type	Constraints	Why needed
`club_id`	`BIGINT`	**PK**, GENERATED IDENTITY	Stable internal identifier
`club_name`	`VARCHAR(150)`	NOT NULL, UNIQUE	Name displayed to users
`club_email`	`VARCHAR(150)`	UNIQUE, NULL	Official club contact/login association if applicable

Final table

```
CLUB
----
club_id              PK
club_name            UNIQUE
club_email           UNIQUE
```

Why generate `club_id`?

The college doesn't have to already provide one.

PostgreSQL generates it:

```
1
2
3
...
```

It is an **internal identifier**, not a claim that the college officially calls the club "17".

Why not use `club_name` as PK?

You could, but it's less stable.

If:

```
TEDxRVCE
```

becomes:

```
TEDx RVCE
```

a name-based primary key would require updating every reference.

With:

```
club_id = 17
club_name = TEDxRVCE
```

the name can change without changing the identity.

Why not description?

Removed.

It isn't necessary for the Activity Point workflow.

Why not status?

Removed from the minimal finalized schema.

Why not created_at?

Removed from the minimal finalized schema.

Those can be added later if the college actually needs them, but they are not necessary for your core functionality.

---

6. EVENT

Purpose

Represents an activity/event that can be displayed to students and associated with official point awards.

We deliberately **do not have an ACTIVITY table**.

`EVENT` is the single entity.

Attributes

Attribute	Type	Constraints	Why needed
`event_id`	`BIGINT`	**PK**, GENERATED IDENTITY	Stable identifier for the event
`event_name`	`VARCHAR(200)`	NOT NULL	Event name
`club_id`	`BIGINT`	FK → `CLUB`, NULL	Identifies organizing club; NULL for non-club activity
`description`	`TEXT`	NULL	Information shown on student Events page
`event_date`	`DATE`	NOT NULL	Event date
`start_time`	`TIME`	NULL	Event timing
`end_time`	`TIME`	NULL	Event timing
`venue`	`VARCHAR(200)`	NULL	Event location
`registration_url`	`TEXT`	NULL	Registration link for student-facing event
`standard_points`	`NUMERIC(6,2)`	NULL, CHECK ≥ 0	Expected/standard points advertised for event
`academic_year`	`VARCHAR(20)`	NOT NULL	Historical/academic-year grouping
`event_status`	`VARCHAR(30)`	NOT NULL	Draft/published/completed/etc.
`created_by`	`VARCHAR(150)`	FK → `USER_ACCOUNT.email`	Identifies Club Lead who created event

Final table

```
EVENT
-----
event_id
event_name
club_id
description
event_date
start_time
end_time
venue
registration_url
standard_points
```

```
academic_year
event_status
created_by
```

Why `event_id`?

Because:

```
Workshop
Workshop
Workshop
```

can happen many times.

The name isn't a reliable unique identifier.

PostgreSQL generates:

```
event_id = 1842
```

automatically.

Why `standard_points` if actual points are in POINT_LEDGER?

They serve different purposes.

`EVENT.standard_points` means:

> What points the event is advertised/expected to carry.

`POINT_LEDGER.points_awarded` means:

> What this particular student was officially awarded.

They can differ.

That distinction is important.

Why `event_status`?

Because the event lifecycle matters:

```
DRAFT
PUBLISHED
COMPLETED
CLOSED
```

The exact status set can be finalized during implementation.

Is `created_by` necessary?

Yes.

The system needs to know which authenticated Club Lead created the event.

---

7. DOCUMENT

Purpose

Represents metadata about every uploaded document.

The actual file lives in object storage.

Attributes

Attribute	Type	Constraints	Why needed
`document_id`	`BIGINT`	**PK**, GENERATED IDENTITY	Internal document reference
`event_id`	`BIGINT`	FK → `EVENT`, NULL	Optional event tag
`document_type`	`VARCHAR(30)`	NOT NULL	Certificate, signed list, photo, etc.
`file_name`	`VARCHAR(255)`	NOT NULL	Original/display filename
`storage_key`	`TEXT`	NOT NULL, UNIQUE	Location of actual object in cloud storage
`mime_type`	`VARCHAR(100)`	NOT NULL	PDF/JPEG/PNG etc.
`file_size_bytes`	`BIGINT`	NOT NULL, CHECK > 0	File validation/storage information
`file_hash`	`CHAR(64)`	NOT NULL	SHA-256 duplicate detection
`uploaded_by`	`VARCHAR(150)`	FK → `USER_ACCOUNT.email`	Who uploaded it
`uploaded_at`	`TIMESTAMPTZ`	NOT NULL	Upload timestamp
`document_status`	`VARCHAR(20)`	NOT NULL	Active/archived/deleted state

Final table

```
DOCUMENT
--------
document_id
event_id
document_type
file_name
storage_key
mime_type
file_size_bytes
file_hash
uploaded_by
```

```
uploaded_at
document_status
```

I am removing `purpose`

This is one of the fields from our earlier design that I would now cut.

Previously:

```
purpose = OFFICIAL / EVIDENCE / ARCHIVE
```

But this is actually derivable from the document's relationships and workflow.

For example:

- linked to a request → request evidence
- linked to a ledger → award evidence
- official signed list → official document type
- unlinked/tagged personal certificate → archive/evidence

Therefore, storing `purpose` introduces another classification that could contradict the actual relationships.

Remove it.

This is exactly the kind of redundant attribute we want to eliminate before drawing the ER diagram.

Why nullable `event_id`?

Because:

```
Certificate → Event X
```

is useful.

But a student can also upload:

```
Old_Certificate.pdf
```

without knowing/tagging an event.

Therefore:

```
event_id NULL
```

is intentional.

8. OCR_BATCH

Purpose

Represents one OCR processing operation on a document.

We need this because OCR results must be **staged before becoming official records**.

Attributes

Attribute	Type	Constraints	Why needed
`ocr_batch_id`	`BIGINT`	**PK**, GENERATED IDENTITY	Identifies one OCR processing operation
`document_id`	`BIGINT`	FK → `DOCUMENT`, NOT NULL	Document being processed
`started_by`	`VARCHAR(150)`	FK → `USER_ACCOUNT.email`	User who initiated OCR
`status`	`VARCHAR(30)`	NOT NULL	Processing/verification/failed/committed state
`started_at`	`TIMESTAMPTZ`	NOT NULL	Processing timestamp
`completed_at`	`TIMESTAMPTZ`	NULL	Completion timestamp
`pages_processed`	`INTEGER`	CHECK ≥ 0	Verification/report statistic
`records_detected`	`INTEGER`	CHECK ≥ 0	Number of OCR rows detected
`valid_records`	`INTEGER`	CHECK ≥ 0	Number successfully validated
`exception_count`	`INTEGER`	CHECK ≥ 0	Number requiring Dean review
`extracted_metadata`	`JSONB`	NULL	Semi-structured OCR document-level output
`error_message`	`TEXT`	NULL	Stores processing failure information

Final table

```
OCR_BATCH
---------
ocr_batch_id
document_id
started_by
status
started_at
completed_at
pages_processed
records_detected
valid_records
```

```
exception_count
extracted_metadata
error_message
```

Why not just put OCR data inside DOCUMENT?

Because a document and an OCR processing operation are different things.

A document can potentially be processed again.

```
DOCUMENT
   │
   ├── OCR_BATCH #1
   │
   └── OCR_BATCH #2
```

This is useful for reprocessing.

---

9. OCR_RECORD

Purpose

Stores each extracted student row before Dean approval.

Attributes

Attribute	Type	Constraints	Why needed
`ocr_record_id`	`BIGINT`	**PK**, GENERATED IDENTITY	Identifies individual extracted row
`ocr_batch_id`	`BIGINT`	FK → `OCR_BATCH`, NOT NULL	Identifies which OCR run produced it
`page_number`	`INTEGER`	NOT NULL	Allows Dean to locate source row in PDF
`row_number`	`INTEGER`	NULL	Identifies row within page/table
`extracted_usn`	`VARCHAR(20)`	NULL	Raw OCR USN
`matched_student_usn`	`VARCHAR(20)`	FK → `STUDENT`, NULL	Student identified after database matching
`extracted_name`	`VARCHAR(150)`	NULL	Raw OCR name
`extracted_points`	`NUMERIC(6,2)`	NULL	Raw extracted points
`confidence_score`	`NUMERIC(5,4)`	CHECK 0–1	OCR confidence
`raw_extraction`	`JSONB`	NULL	Raw/semi-structured OCR information
`validation_status`	`VARCHAR(30)`	NOT NULL	Valid/mismatch/exception/etc.

Attribute	Type	Constraints	Why needed
`validation_message`	`TEXT`	NULL	Explains exception

Final table

```
OCR_RECORD
----------
ocr_record_id
ocr_batch_id
page_number
row_number
extracted_usn
matched_student_usn
extracted_name
extracted_points
confidence_score
raw_extraction
validation_status
validation_message
```

Why both `extracted_usn` and `matched_student_usn`?

This is **not redundant**.

Suppose OCR reads:

```
1RV22CS001
```

but database matching determines:

```
1RV22CS010
```

We need to preserve both:

```
what OCR saw
vs
what database matching found
```

This is crucial for human verification and auditability.

---

10. POINT_REQUEST

Purpose

Represents a request for an Activity Point action.

This is deliberately separate from the official ledger.

Attributes

Attribute	Type	Constraints	Why needed
`request_id`	`BIGINT`	**PK**, GENERATED IDENTITY	Identifies individual request
`usn`	`VARCHAR(20)`	FK → `STUDENT`, NOT NULL	Student making/requested for
`event_id`	`BIGINT`	FK → `EVENT`, NOT NULL	Event involved
`request_type`	`VARCHAR(30)`	NOT NULL	NEW_ACTIVITY / MISSING_FROM_LIST / CORRECTION
`claimed_points`	`NUMERIC(6,2)`	CHECK ≥ 0	Points requested/proposed
`reason`	`TEXT`	NOT NULL	Student's explanation
`submitted_by`	`VARCHAR(150)`	FK → `USER_ACCOUNT.email`	Who submitted request
`correction_of_ledger_id`	`BIGINT`	FK → `POINT_LEDGER`, NULL	Existing award being corrected
`status`	`VARCHAR(30)`	NOT NULL	Workflow state
`club_reviewed_by`	`VARCHAR(150)`	FK → `USER_ACCOUNT.email`, NULL	Club reviewer
`club_reviewed_at`	`TIMESTAMPTZ`	NULL	Club review time
`club_review_comment`	`TEXT`	NULL	Club decision/comment
`dean_decided_by`	`VARCHAR(150)`	FK → `USER_ACCOUNT.email`, NULL	Final Dean reviewer
`dean_decided_at`	`TIMESTAMPTZ`	NULL	Dean decision time
`dean_decision_comment`	`TEXT`	NULL	Dean decision/comment
`created_at`	`TIMESTAMPTZ`	NOT NULL	Request creation
`updated_at`	`TIMESTAMPTZ`	NOT NULL	Last workflow update

Final table

```
POINT_REQUEST
-------------
request_id
usn
event_id
request_type
claimed_points
```

```
reason
submitted_by
correction_of_ledger_id
status
club_reviewed_by
club_reviewed_at
club_review_comment
dean_decided_by
dean_decided_at
dean_decision_comment
created_at
updated_at
```

Why keep the club-review fields?

Because Club Lead is a genuine workflow stage.

We need to know:

```
Who reviewed it?
When?
What did they say?
```

Same for Dean.

These aren't merely audit-log details because the application needs to display the current request workflow directly.

Why `correction_of_ledger_id`?

A correction must identify exactly which existing award is being challenged.

---

11. POINT_REQUEST_DOCUMENT

Purpose

Allows multiple supporting documents for one request.

Example:

```
Request 501
 ├── Certificate
 ├── Attendance photograph
 └── Supporting photograph
```

Attributes

Attribute	Type	Constraints	Why needed
`request_id`	`BIGINT`	FK → `POINT_REQUEST`	Request side
`document_id`	`BIGINT`	FK → `DOCUMENT`	Document side
`attached_at`	`TIMESTAMPTZ`	NOT NULL	When document was attached

Primary key

```
(request_id, document_id)
```

Anything redundant?

No.

This is a true many-to-many relationship.

---

12. POINT_LEDGER

Purpose

The official academic Activity Point record.

This is the most important table in the system.

Attributes

Attribute	Type	Constraints	Why needed
`ledger_id`	`BIGINT`	**PK**, GENERATED IDENTITY	Identifies individual award
`usn`	`VARCHAR(20)`	FK → `STUDENT`, NOT NULL	Student receiving points
`event_id`	`BIGINT`	FK → `EVENT`, NOT NULL	Event for which points were awarded
`points_awarded`	`NUMERIC(6,2)`	NOT NULL, CHECK ≥ 0	Actual official points
`award_date`	`DATE`	NOT NULL	Official award date
`source_type`	`VARCHAR(30)`	NOT NULL	BULK_APPROVAL / INDIVIDUAL_CLAIM / MANUAL_CORRECTION
`source_request_id`	`BIGINT`	FK → `POINT_REQUEST`, NULL	Links award to approved student request
`source_ocr_batch_id`	`BIGINT`	FK → `OCR_BATCH`, NULL	Links bulk award to OCR processing

Attribute	Type	Constraints	Why needed
`approved_by`	`VARCHAR(150)`	FK → `USER_ACCOUNT.email`	Dean who authorized award
`approved_at`	`TIMESTAMPTZ`	NOT NULL	Approval timestamp
`status`	`VARCHAR(20)`	NOT NULL	ACTIVE / SUPERSEDED / REVOKED
`supersedes_ledger_id`	`BIGINT`	FK → `POINT_LEDGER`, NULL	Links corrected award to previous award

Final table

```
POINT_LEDGER
------------
ledger_id
usn
event_id
points_awarded
award_date
source_type
source_request_id
source_ocr_batch_id
approved_by
approved_at
status
supersedes_ledger_id
```

I would remove `created_at`

It was in the previous design, but I don't think it is necessary.

`approved_at` already records the important official timestamp, and the ledger record should be created as part of that approval transaction.

So:

> **CUT `created_at` from POINT_LEDGER.**

---

13. Why both `source_request_id` and `source_ocr_batch_id`?

This looks redundant at first, but it isn't.

There are two fundamentally different award sources.

Bulk official document

```
Signed PDF
 ↓
```

```
OCR_BATCH
 ↓
POINT_LEDGER
```

So:

```
source_ocr_batch_id
```

is populated.

Individual claim

```
Student request
 ↓
Dean approval
 ↓
POINT_LEDGER
```

So:

```
source_request_id
```

is populated.

Correction

Can reference:

```
source_request_id
```

and:

```
supersedes_ledger_id
```

This gives us proper traceability without a dangerous polymorphic `source_id`.

---

14. POINT_LEDGER_DOCUMENT

Purpose

Connects official awards to supporting documents.

This is required because:

> One award can have multiple documents.

and:

> One official document can support multiple awards.

Attributes

Attribute	Type	Constraints	Why needed
`ledger_id`	`BIGINT`	FK → `POINT_LEDGER`	Award side
`document_id`	`BIGINT`	FK → `DOCUMENT`	Document side
`linked_at`	`TIMESTAMPTZ`	NOT NULL	When relationship was created
`linked_by`	`VARCHAR(150)`	FK → `USER_ACCOUNT.email`	Who established relationship

Primary key

```
(ledger_id, document_id)
```

Why absolutely necessary?

Because this would not work correctly:

```
POINT_LEDGER
------------
document_id
```

One award could have:

```
Certificate
Attendance photo
Official signed PDF
```

And one signed PDF could support:

```
500 students
```

Therefore this is a genuine many-to-many relationship.

---

15. AUDIT_LOG

Purpose

Provides long-term accountability.

Attributes

Attribute	Type	Constraints	Why needed
`audit_id`	`BIGINT`	**PK**, GENERATED IDENTITY	Individual audit entry
`user_email`	`VARCHAR(150)`	FK → `USER_ACCOUNT.email`	Who performed action
`action`	`VARCHAR(100)`	NOT NULL	What happened
`entity_type`	`VARCHAR(50)`	NOT NULL	Which kind of object
`entity_id`	`BIGINT`	NOT NULL	Which record
`old_values`	`JSONB`	NULL	Previous values where relevant
`new_values`	`JSONB`	NULL	New values where relevant
`ip_address`	`INET`	NULL	Security/audit information
`created_at`	`TIMESTAMPTZ`	NOT NULL	When action happened

Final table

```
AUDIT_LOG
---------
audit_id
user_email
action
entity_type
entity_id
old_values
new_values
ip_address
created_at
```

Why `old_values/new_values`?

Suppose Dean changes:

```
10 points → 15 points
```

The audit log can preserve the before/after state.

`JSONB` is appropriate because different entity types have different fields.

---

16. The one view: STUDENT_POINT_SUMMARY

This is **not a table**.

It can calculate:

```
USN
Name
Required Points
Earned Points
Remaining Points
Target Reached
```

from:

```
STUDENT
+
POINT_REQUIREMENT
+
POINT_LEDGER
```

Why no `total_points` in STUDENT?

Because that would duplicate ledger information.

If:

```
POINT_LEDGER
10 + 15 + 20
```

then total is:

```
45
```

There is no reason to separately store:

```
STUDENT.total_points = 45
```

That could become inconsistent.

---

17. Final exact table count and attribute count

With the redundant fields removed, I get:

#	Table	Attributes
1	`DEPARTMENT`	**2**
2	`STUDENT`	**8**

#	Table	Attributes
3	`USER_ACCOUNT`	**9**
4	`POINT_REQUIREMENT`	**2**
5	`CLUB`	**3**
6	`EVENT`	**13**
7	`DOCUMENT`	**11**
8	`OCR_BATCH`	**12**
9	`OCR_RECORD`	**12**
10	`POINT_REQUEST`	**16**
11	`POINT_REQUEST_DOCUMENT`	**3**
12	`POINT_LEDGER`	**12**
13	`POINT_LEDGER_DOCUMENT`	**4**
14	`AUDIT_LOG`	**9**
	TOTAL	**126 attributes/columns**

For the two junction tables, the first two columns form the composite primary key.

---

18. Fields I specifically removed

This is important because we've gone through several iterations.

Removed tables

❌ `ACTIVITY`

`EVENT` is sufficient.

❌ `EVENT_PARTICIPATION`

The system isn't an attendance-management system.

❌ `STUDENT_CLUB_MEMBERSHIP`

Not required for Activity Point calculation.

❌ `AICTE_CATEGORY`

Not required by your finalized requirements.

Removed attributes

From `STUDENT`

❌ `total_points`

Derived from ledger.

From `CLUB`

❌ `description`

Not required.

❌ `status`

Not required for core functionality.

❌ `created_at`

Not required for core functionality.

From `DOCUMENT`

❌ `purpose`

This can be determined through the document's relationships/workflow.

From `POINT_LEDGER`

❌ `created_at`

`approved_at` provides the relevant official timestamp.

From Club accountability

❌ `open_issue_count`

Must **not** be stored.

It is calculated from:

```
POINT_REQUEST.status
```

This is extremely important.

---

19. One thing I would NOT cut despite looking redundant

There are several places where you might be tempted to remove something.

`STUDENT.email`

Keep it.

Even though `USER_ACCOUNT.email` exists, the student record needs its academic/contact email independently.

`EVENT.standard_points`

Keep it.

It represents the advertised/standard event value.

It is not necessarily the same as the student's actual award.

```
EVENT.standard_points
        ≠
POINT_LEDGER.points_awarded
```

`OCR_RECORD.extracted_usn`

Keep it.

It records what OCR actually saw.

```
extracted_usn
      ↓
database matching
      ↓
matched_student_usn
```

These are deliberately different.

`POINT_LEDGER.source_request_id`

Keep it.

It provides direct traceability from an approved individual request to the resulting official award.

`POINT_LEDGER.source_ocr_batch_id`

Keep it.

It provides direct traceability from a bulk OCR import to every resulting official award.

`POINT_LEDGER.supersedes_ledger_id`

Keep it.

This is what allows correction without destroying historical information.

`DOCUMENT.event_id`

Keep it.

It enables the feature you specifically requested:

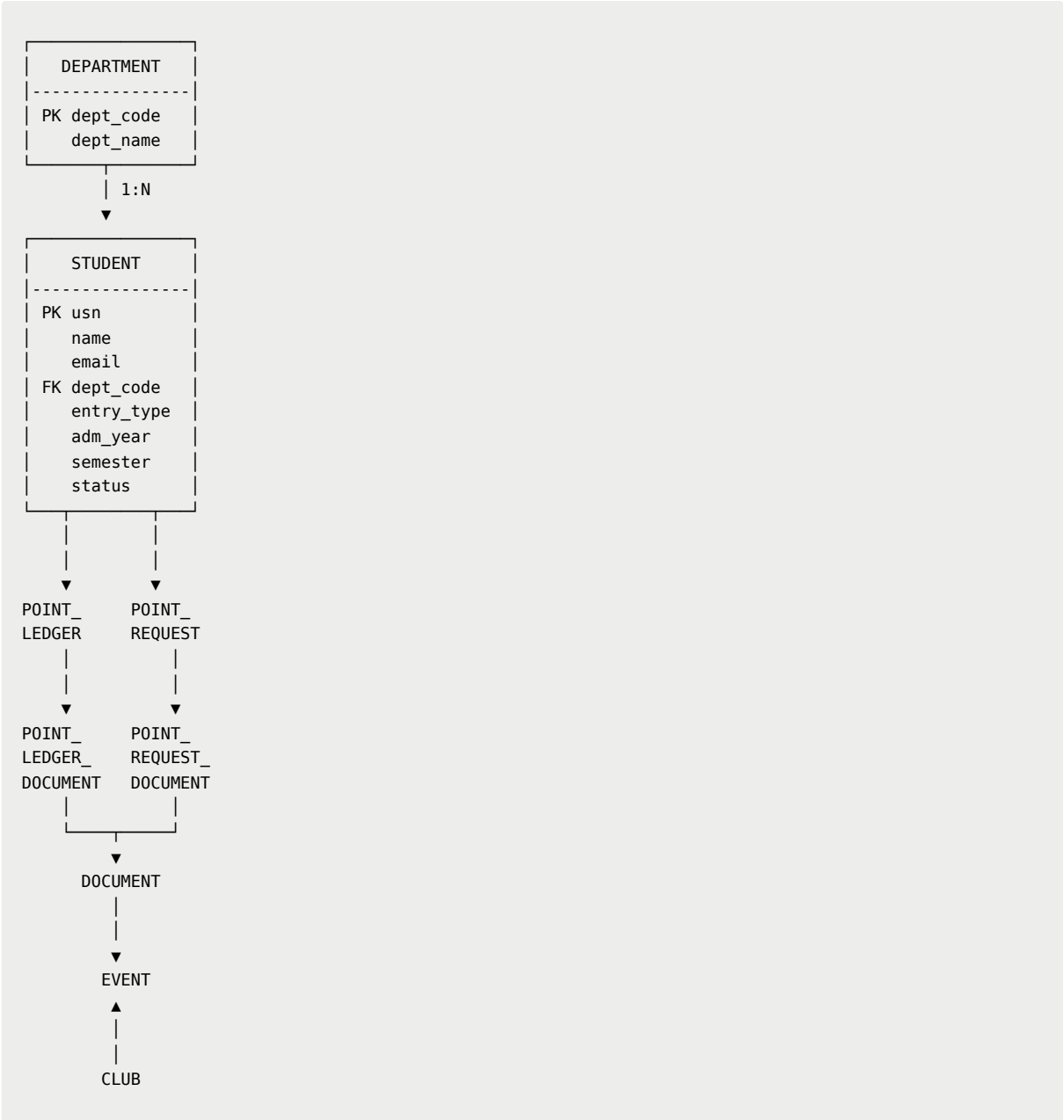
> Student uploads a certificate and tags it to an event.

It also allows Dean/HOD/Club vaults to organize documents by event.

---

20. The exact ER diagram you should draw

At the highest level:



Then:

```
DOCUMENT
    |
    ▼
OCR_BATCH
    |
    ▼
OCR_RECORD
```

And:

```
USER_ACCOUNT
      |
      ▼
 AUDIT_LOG
```

---

21. Cardinalities to put on your ER diagram

These are the ones you should use:

```
DEPARTMENT 1 ——— N STUDENT

DEPARTMENT 1 ——— N USER_ACCOUNT
                 (HOD association)

CLUB 1 ——— N EVENT

USER_ACCOUNT 1 ——— N EVENT
                 (created_by)

STUDENT 1 ——— N POINT_LEDGER

EVENT 1 ——— N POINT_LEDGER

STUDENT 1 ——— N POINT_REQUEST

EVENT 1 ——— N POINT_REQUEST

POINT_REQUEST 1 ——— N POINT_REQUEST_DOCUMENT

DOCUMENT 1 ——— N POINT_REQUEST_DOCUMENT

POINT_LEDGER 1 ——— N POINT_LEDGER_DOCUMENT

DOCUMENT 1 ——— N POINT_LEDGER_DOCUMENT

DOCUMENT 1 ——— N OCR_BATCH

OCR_BATCH 1 ——— N OCR_RECORD

USER_ACCOUNT 1 ——— N DOCUMENT
```

```
                  (uploaded_by)

USER_ACCOUNT 1 ─── N AUDIT_LOG
```

And `DOCUMENT.event_id` gives:

```
EVENT 1 ─── N DOCUMENT
```

---

22. The most important conceptual relationships

Your ER diagram should visually make these three separate paths obvious:

Official bulk awarding

```
EVENT
  ↓
DOCUMENT
  ↓
OCR_BATCH
  ↓
OCR_RECORD
  ↓
POINT_LEDGER
```

Individual claim

```
STUDENT
  ↓
POINT_REQUEST
  ↓
DOCUMENT
  ↓
CLUB / DEAN verification
  ↓
POINT_LEDGER
```

Already-awarded evidence

```
STUDENT
  ↓
DOCUMENT
  ↓
EVENT
  ↓
Existing POINT_LEDGER
```

No new points are created.

23. One final decision about the 14-table model

I would **freeze the database at these 14 tables** for your ER diagram.

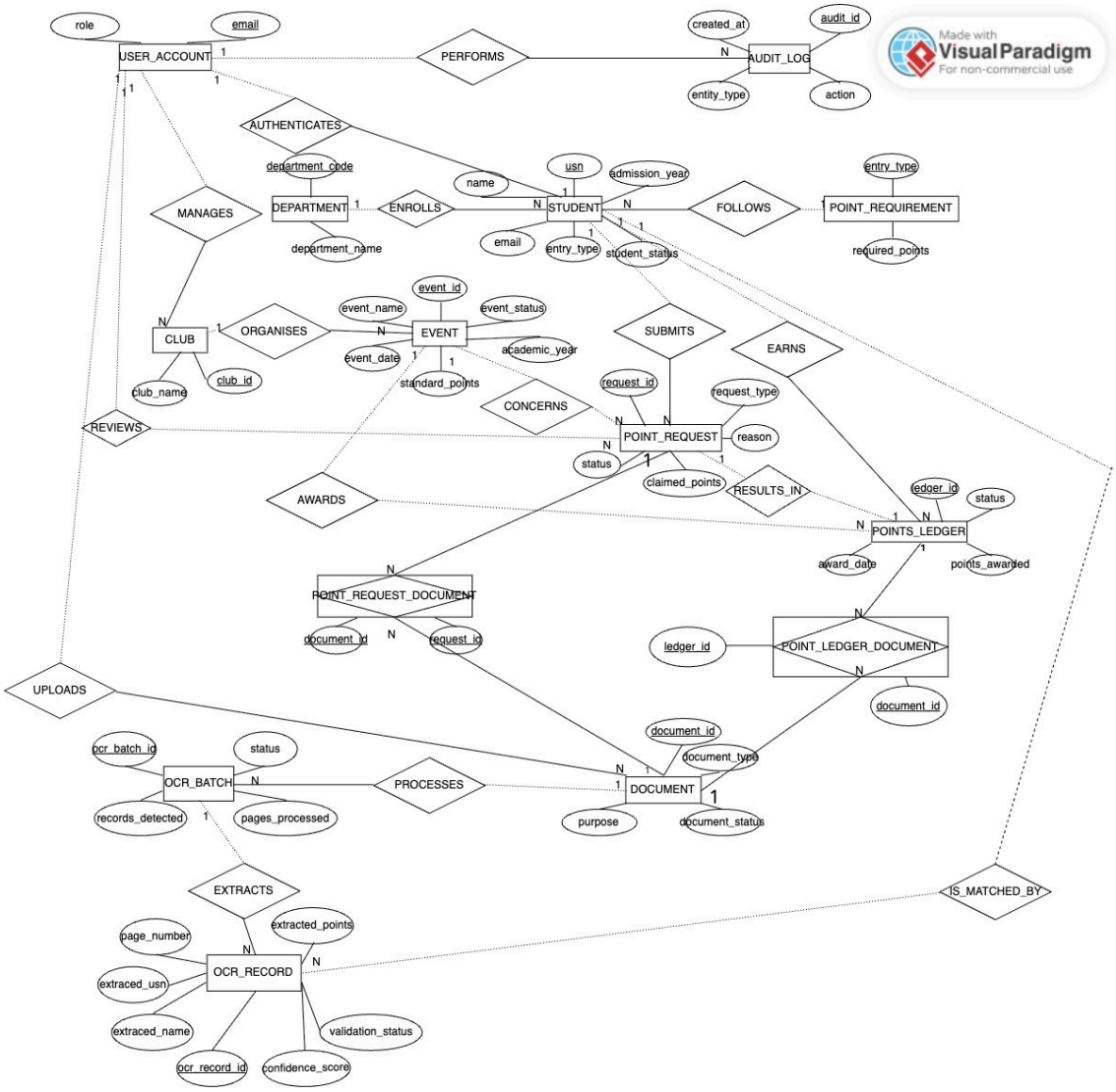
The important thing is that the 14 tables are not there to artificially make the project "complex." Each represents a genuinely different concept:

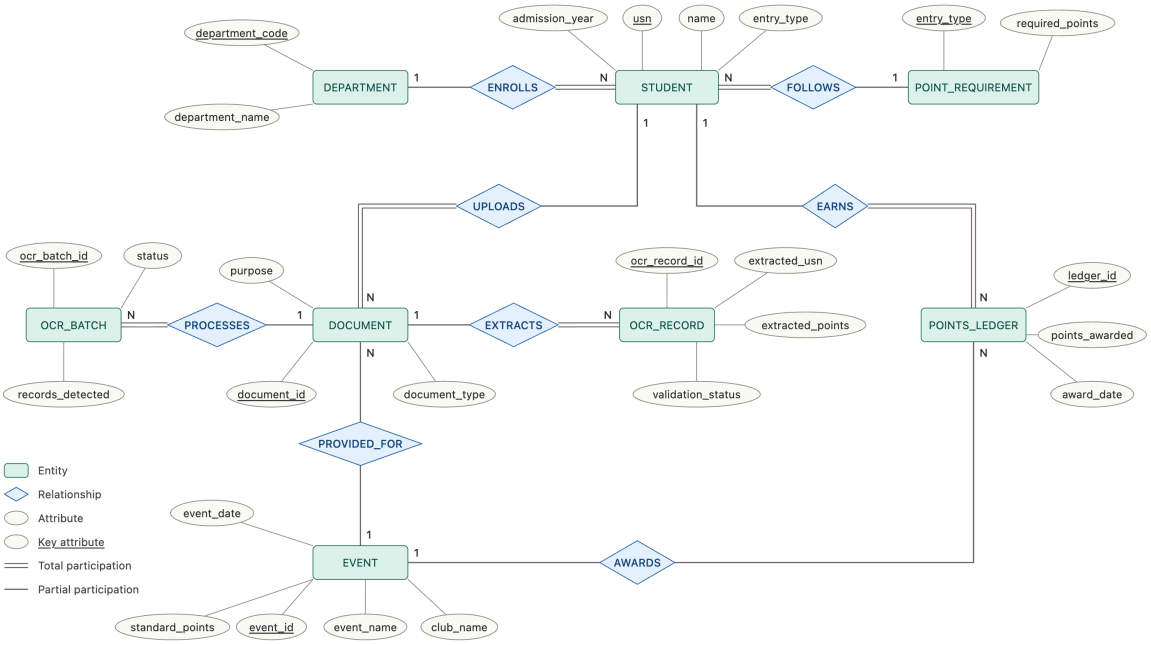
Table	Represents
`DEPARTMENT`	Organizational department
`STUDENT`	Student academic identity
`USER_ACCOUNT`	Login/security identity
`POINT_REQUIREMENT`	Point target rules
`CLUB`	Organizing club
`EVENT`	Student-facing activity/event
`DOCUMENT`	Uploaded evidence/official file metadata
`OCR_BATCH`	One OCR processing operation
`OCR_RECORD`	One extracted/staged OCR row
`POINT_REQUEST`	Request for an Activity Point action
`POINT_REQUEST_DOCUMENT`	Evidence attached to a request
`POINT_LEDGER`	Official awarded points
`POINT_LEDGER_DOCUMENT`	Evidence supporting an official award
`AUDIT_LOG`	Historical accountability

That separation is justified by the actual workflow, especially the crucial distinction between **event → request → award → document → OCR processing**.

For your ER sketch, use these 14 tables and these final attributes. I would not add `ACTIVITY`, `EVENT_PARTICIPATION`, `AICTE_CATEGORY`, `STUDENT_CLUB_MEMBERSHIP`, `STUDENT.total_points`, `CLUB.issue_count`, or `DOCUMENT.purpose`.

▼ ER diagram


role
email
USER_ACCOUNT
PERFORMS
created_at
audit_id
AUDIT_LOG
entity_type
action
Made with Visual Paradigm For non-commercial use
AUTHENTICATES
department_code
name
usn
admission_year
entry_type
MANAGES
DEPARTMENT
ENROLLS
STUDENT
FOLLOWS
POINT_REQUIREMENT
department_name
email
entry_type
student_status
required_points
event_id
event_name
event_status
CLUB
ORGANISES
EVENT
SUBMITS
EARNS
academic_year
event_date
club_name
club_id
standard_points
CONCERNS
request_id
request_type
REVIEWS
POINT_REQUEST
reason
status
claimed_points
RESULTS_IN
AWARDS
ledger_id
status
POINTS_LEDGER
award_date
points_awarded
POINT_REQUEST_DOCUMENT
document_id
request_id
ledger_id
POINT_LEDGER_DOCUMENT
document_id
UPLOADS
document_id
document_type
ocr_batch_id
status
OCR_BATCH
PROCESSES
DOCUMENT
records_detected
pages_processed
purpose
document_status
EXTRACTS
IS_MATCHED_BY
extracted_points
page_number
OCR_RECORD
extraced_usn
extraced_name
validation_status
ocr_record_id
confidence_score


department_code
DEPARTMENT
department_name
1
ENROLLS
admission_year
usn
name
entry_type
N
STUDENT
N
FOLLOWS
1
entry_type
required_points
POINT_REQUIREMENT
1
1
UPLOADS
EARNS
ocr_batch_id
status
OCR_BATCH
records_detected
N
PROCESSES
purpose
1
DOCUMENT
document_id
document_type
N
1
EXTRACTS
N
ocr_record_id
extracted_usn
OCR_RECORD
extracted_points
validation_status
N
ledger_id
POINTS_LEDGER
points_awarded
award_date
N
PROVIDED_FOR
Entity
Relationship
Attribute
Key attribute
Total participation
Partial participation
event_date
1
EVENT
standard_points
event_id
event_name
club_name
1
AWARDS